

Anatomie d'un patch — M-EPI-000

Lire un correctif comme une hypothèse

Ted "Tedsig42" Kouhouenou ted.sig42@tutamail.com

Summary: Apprendre à lire un diff de sécurité, à identifier l'hypothèse ajoutée par le développeur et à formuler ce que le correctif ne couvre pas. Ce module est la porte d'entrée de la piscine : aucun exploit, seulement de la lecture rigoureuse.

Prerequisites : Shell Linux, git, bases de C

Version: 1.0

Contents

I Instructions **2**

II Instructions Communes **5**

II.1 Structure du Dépôt 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide 5

II.3 Processus de Soumission 6

II.4 Règle des Fichiers Supplémentaires Interdits 7

II.5 Format du Rapport 7

II.6 Cibles Autorisées Uniquement 8

II.7 Citez Toutes Vos Sources 8

III Instructions sur l'IA **10**

III.1 Aperçu de la Politique 10

III.2 Utilisations Autorisées 10

III.3 Utilisations Interdites 11

III.4 Responsabilité Personnelle 11

III.5 Citation Obligatoire de l'IA 11

III.6 Évaluation par les Pairs et l'IA 12

III.7 Rappel Éthique 13

IV Avant-propos **14**

V Aperçu du module **15**

VI Exercise 00 **16**

VII Exercise 01 **18**

VIII Exercise 02 **20**

IX Exercise 03 **22**

X Rendu et évaluation **23**

XI Ressources par exercice **24**

XII Sources et prolongements **26**

Chapter I

Instructions

- Ce module se travaille en une journée, seul ou en binôme.
- Tous les exercices se font sur une copie locale de sources ou sur un dépôt jouet.
- Aucun test sur une cible réelle, aucun exploit, aucun noyau de production.
- Citez toutes vos sources, y compris les outils d'IA si vous en utilisez.
- Les rendus sont évalués par les pairs avec la grille fournie en fin de sujet.

Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex: `tedsig42 @tedsig42`).

Return — selon le type de fichier

Type de fichier	Valeur de Return
C (`.c`)	`return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé)
Python (`.py`)	`0` (succès), `1` (erreur), `N/A` (analyse, script sans exit)
Shell (`.sh`)	`exit 0` (succès), `exit 1` (erreur)
Markdown (`.md`)	`N/A` — les fichiers d'analyse n'ont pas de valeur de retour
Exploit C/Python	`shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept)
Binaire (`.bin`)	`N/A` — pas d'en-tête pour les fichiers binaires

Exemples par langage

C (`.c`) :

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

Python (`.py`) :

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

Shell (`.sh`) :

```shell
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
#
# Return: exit 0
```

Markdown (`.md`) — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

Chapter II

Instructions Communes

II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

II.6 Cibles Autorisées Uniquement

① Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

Chapter III

Instructions sur l'IA

III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

Exemples

Python :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

C :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

Markdown (rapport) :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

Chapter IV

Avant-propos

> “Every patch encodes an assumption about what can go wrong. Your job is to find the assumption — and watch what it mutates into.”

Un correctif de sécurité n’est pas une vérité. C’est une phrase qu’un développeur écrit sur le passé : *le bug était ici, sur ce chemin, sous cette condition*. Cette phrase est souvent vraie. Elle est rarement complète.

Votre travail commence donc avant l’exploitation : lire le diff, nommer l’hypothèse, puis chercher les endroits où la même hypothèse n’a pas été vérifiée. Un patch qui ajoute une vérification dans une fonction nous dit aussi, en creux, quelles fonctions voisines n’ont pas reçu cette vérification.



C

e module ne demande pas de comprendre le noyau Linux en profondeur. Il demande de savoir répondre à trois questions devant un diff : quelle ligne a changé, quel invariant est supposé vrai, et quel chemin n’est pas couvert ?

Chapter V

Aperçu du module

Ex	Titre	Difficulté	Points	
00	Anatomie d'un diff	Facile	2	
01	Le patch réel CVE-2024-1085	Moyen	4	
02	De CVE-2023-4244 à CVE-2024-1085	Moyen	4	
03	Autopsie d'un patch	Difficile	6	
	Total		**16**	

Chapter VI

Exercise 00

	Exercise 00
Anatomie d'un diff	
Turn-in directory : `ex00/`	
Files to turn in : `patch-anatomy.md assumptions.md`	
Allowed functions : `git, grep, diff, un éditeur de texte`	

Un diff contient quatre informations utiles : ce qui est supprimé, ce qui est ajouté, la fonction concernée et le contexte. La cinquième information — l'hypothèse — n'est jamais écrite explicitement.

Mission

Créez un petit dépôt local avec un fichier C contenant une fonction `check_len` qui vérifie une longueur. Faites deux commits : une version « vulnerable » et une version « corrigée ». Utilisez un diff minimal, par exemple :

```
- if (len > MAX)
+ if (len >= MAX)
      return (-1);
```

Puis analysez le diff dans `patch-anatomy.md` :

1. Quelle ligne est supprimée ? Quelle ligne est ajoutée ?
2. Quelle fonction est modifiée ?
3. Quelle hypothèse le développeur encode-t-il ? (Que se passait-il avant ? Que se passe-t-il après ?)
4. Quel appelant ou quel cas limite n'est pas couvert par ce diff ?
5. Le changement est-il une correction, un durcissement, ou les deux ?

Dans `assumptions.md`, écrivez une seule phrase par hypothèse, au format : *« Le développeur suppose que … »*. Ajoutez pour chacune un test qui pourrait rendre l'hypothèse fausse.



Un bon diff ne se lit pas seulement de haut en bas. Lisez aussi les lignes qui *ne changent pas* : ce sont elles qui portent souvent l'hypothèse implicite.

Chapter VII

Exercise 01

	Exercise 01
Le patch réel CVE-2024-1085	
Turn-in directory : `ex01/`	
Files to turn in : `cve-2024-1085.md`	
Allowed functions : `git, grep, un éditeur de texte, accès à NVD`	

Le correctif de CVE-2024-1085 est une seule ligne. C'est un excellent cas pour apprendre à lire lentement.

```
- if (!nft_is_active(net, ext))
+ if (!nft_is_active_next(net, ext))
      continue;
```

Le commit amont est `b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7`. Il est décrit par NVD comme un use-after-free dans `nft_setelem_catchall_deactivate()` : la fonction vérifiait l'état dans la génération courante au lieu de la génération suivante avant de libérer l'élément.

Mission

Dans `cve-2024-1085.md`, répondez :

1. Pourquoi `nft_is_active()` et `nft_is_active_next()` ne sont-ils pas équivalents dans une transaction nf_tables ?

2. Que signifie « génération » dans le modèle de transaction de nf_tables ? Pourquoi un élément peut-il être actif dans une génération et inactif dans l'autre ?

3. Quelle est la conséquence de vérifier la mauvaise génération avant un `kfree()` ?

4. Le patch corrige-t-il la cause ou seulement un symptôme ? Justifiez.



N

e cherchez pas à reproduire l'exploit. L'objectif est de comprendre le modèle de génération et l'asymétrie entre le chemin nominal et le chemin de nettoyage. Un noyau de production n'est pas un laboratoire.

Sources

- NVD, CVE-2024-1085 : https://nvd.nist.gov/vuln/detail/CVE-2024-1085
- Commit amont : https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7
- Patch commenté sur Ubuntu : https://lists.ubuntu.com/archives/kernel-team/2024-February/149036.html

Chapter VIII

Exercise 02

	Exercise 02
De CVE-2023-4244 à CVE-2024-1085	
Turn-in directory : `ex02/`	
Files to turn in : `mutation-map.md`	
Allowed functions : `git, grep, un éditeur de texte`	

Le papier décrit une mutation : le correctif de CVE-2023-4244 retire la marque « busy » et l'API de batch GC (`a2dd0233cbc4`). Cinq mois plus tard, CVE-2024-1085 apparaît dans le même sous-système. Le second bug n'est pas une copie du premier : c'est une faiblesse préexistante qui était masquée par la garde supprimée.

Mission

Construisez `mutation-map.md` avec trois colonnes :

Étape	**Question**	**Votre réponse**
Patch A	Que retire le correctif de CVE-2023-4244 ?	
Hypothèse A	Quel invariant la garde supprimée protégeait-elle ?	
Patch B	Que change le correctif de CVE-2024-1085 ?	
Hypothèse B	Quelle hypothèse le nouveau test encode-t-il ?	
Mutation	Pourquoi parle-t-on de *incomplete-fix variant* et non d'*architectural scatter* ?	

Ajoutez une conclusion de dix lignes maximum : quel réflexe d'audit ce cas vous donne-t-il pour un prochain patch dans le même fichier ?



Relisez la section 2 et la section 6.3 du papier. La distinction importante est celle entre « le patch a manqué un jumeau » et « le patch a supprimé une garde qui cachait un autre bug ».

Chapter IX

Exercise 03

	Exercise 03
Autopsie d'un patch	
Turn-in directory : `ex03/`	
Files to turn in : `autopsy.md`	
Allowed functions : `git, grep, un éditeur de texte`	

Choisissez un correctif public dans un sous-système que vous pouvez lire (nf_tables, eBPF, io_uring, ou un projet utilisateur). Le correctif doit être petit : moins de cinquante lignes ajoutées.

Mission

Produisez `autopsy.md` avec exactement six sections :

1. **Identité** : CVE, commit, fichier, fonction.
2. **Diff** : les lignes ajoutées et supprimées, recopiées proprement.
3. **Hypothèse** : ce que le développeur suppose vrai après le patch.
4. **Chemin non couvert** : un appelant, un chemin d'abandon ou une fonction voisine qui n'a pas reçu le même test.
5. **Test de l'hypothèse** : une commande `grep` qui vérifie votre hypothèse.
6. **Verdict** : patch complet, patch local, ou patch qui déplace le problème. Justifiez en cinq lignes.

La soutenance dure cinq minutes : présentez le diff, l'hypothèse et le chemin non couvert. Le pair doit pouvoir refaire votre commande et obtenir le même résultat.

Chapter X

Rendu et évaluation

Votre dépôt doit contenir uniquement :

```
M-EPI-000-patch-anatomy/
|-- ex00/
|   |-- patch-anatomy.md
|   +-- assumptions.md
|-- ex01/
|   +-- cve-2024-1085.md
|-- ex02/
|   +-- mutation-map.md
+-- ex03/
    +-- autopsy.md
```

Grille

Critère	**Attendu**	**Points**
Lecture du diff	Lignes, fonction et contexte correctement identifiés	4
Hypothèse	Formulée comme une phrase testable, pas comme un slogan	4
Chemin non couvert	Appuyé par une commande ou un appelant précis	4
Rigueur	Sources citées, pas d'exploit, pas d'affirmation non vérifiée	4
Soutenance	Explication claire en cinq minutes	4

Chapter XI

Ressources par exercice

Exercice 00 — Anatomie d'un diff

- `git diff` : https://git-scm.com/docs/git-diff
- Pro Git, chapitre 2 : https://git-scm.com/book/fr/v2
- Papier, section 4 : chaque ligne ajoutée encode une hypothèse.
- À chercher : `git diff -unified=0`, `diff -u`.

Exercice 01 — CVE-2024-1085

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2024-1085
- Commit `b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7` : https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=b1db244ffd041a49ecc9618e8feb6b5c1afcdaa7
- Patch commenté : https://lists.ubuntu.com/archives/kernel-team/2024-February/149036.html
- nf_tables : https://www.kernel.org/doc/html/latest/networking/nf_tables.html
- À chercher : `nft_is_active`, `nft_is_active_next`, `nft_setelem_catchall_deactivate`.

Exercice 02 — De CVE-2023-4244 à CVE-2024-1085

- NVD, CVE-2023-4244 : https://nvd.nist.gov/vuln/detail/CVE-2023-4244
- NVD, CVE-2024-1085 : https://nvd.nist.gov/vuln/detail/CVE-2024-1085
- Papier, sections 2 et 6.3.
- À chercher : `a2dd0233cbc4`, `nft_setelem_catchall`, `nft_trans_commit_list_add_tail`.

Exercice 03 — Autopsie d'un patch

- NVD : https://nvd.nist.gov/
- Noyau Linux : https://git.kernel.org/
- Optionnel : Universal Ctags : https://docs.ctags.io/
- Optionnel : Semgrep : https://semgrep.dev/docs/
- À chercher : `git show <commit>`, `grep -RIn`, `grep '^+' fix.patch`.

Chapter XII

Sources et prolongements

- Phrack, *Security is failure: Every Patch Is a Guess*, sections 1, 2, 4 et 6.3.
- NVD, CVE-2024-1085 et CVE-2023-4244.
- Documentation nf_tables : https://www.kernel.org/doc/html/latest/networking/nf_tables.html
- Guide `git diff` : https://git-scm.com/docs/git-diff